



Análisis de Grandes Volúmenes de Datos

POSGRADOS

Introducción a Big Data con PySpark

Agenda



1. Conceptos Generales
PySpark
2. Manipulación de datos
básica con PySpark
 1. Dataframes
 2. Operaciones en
registros/columnas con
PySpark
 3. SQL en Spark
3. Conclusiones

Conceptos Generales PySpark



Conceptos clave de Apache Spark

- Apache Spark (en lo futuro Spark) es un conjunto de herramientas para el procesamiento de Big Data desarrollado por Apache
 - Basado en una arquitectura cliente - esclavo
 - Soporte de múltiples lenguajes de programación (Python, Java, Scala, R)
 - Implementa el soporte de estructura de datos distribuidas (dataframes Spark)
 - Soporte de operadores propios o a través de lenguaje de consulta SQL
 - Bibliotecas de aprendizaje máquina para el cómputo distribuido
- **Nota:** en el curso se estará trabajando con la versión asociada al lenguaje Python, el cual se denomina **PySpark**

Estructura de datos

- Apache Spark ofrece dos tipos de estructuras de datos con soporte para el procesamiento distribuido
 - RDD (Resilient Distributed Datasets): abstracción más básica de Spark, introducida en las primeras versiones. No incluye información estructural sobre los datos (nombre de columnas, tipos de datos)
 - Dataframes: introducido a partir de la versión 1.3 (2015). Estructura de datos organizada en columnas, ofreciendo operadores como filtrado, agrupación, entre otros. Soporta operadores nativos y a través de SQL
- Las estructuras de datos son inmutables

Particiones (Partitions)

- Para que cada ejecutor pueda ejecutar operaciones en paralelo, Spark divide los datos en fragmentos llamados particiones
 - Una partición es una colección de datos (instancias) que está ubicada físicamente en una computadora del cluster
 - Si se tiene una única partición de un conjunto de datos, Spark podrá realizar cómputo paralelo sobre esa partición (a pesar de que se tengan múltiples ejecutores)
 - Si se tienen múltiples particiones pero un solo ejecutor, Spark podrá realizar cómputo paralelo sobre las particiones sobre los recursos de cómputo de la computadora anfitriona

Particiones en Dataframes

- Es importante señalar que con los Dataframes de Spark no es posible manipular las particiones de manera manual o individual. En lugar de ello, se especifican **transformaciones** de alto nivel de los datos en las particiones físicas, y Spark determina como se ejecutará este trabajo en el clúster

Transformaciones

- Debido a que los Dataframes son estructuras inmutables, se tienen que especificar instrucciones (transformaciones) que definen las modificaciones que se desean realizar sobre un Dataframe, las cuales se ejecutarán hasta especificar una acción
 - Las transformaciones son lazy, por lo que no se ejecutan inmediatamente (se almacena como un plan de ejecución), hasta que se aplique una acción, como `count()`, `collect()`, `show()`
 - Lo anterior permite realizar optimización de ejecución por parte de Spark

Transformaciones

- Ejemplo en el cual se definen dos transformaciones – read(), filter() – y una acción – count()
- En este ejemplo, las transformaciones se agregan al plan y no son ejecutadas hasta que se lanza la acción

```
>>> strings = spark.read.text("../README.md")
>>> filtered = strings.filter(strings.value.contains("Spark"))
>>> filtered.count()
20
```

Tipos de transformaciones

- Existen dos tipos de transformaciones en Spark:
 - Transformaciones estrechas (narrow transformations)
 - Cada partición de entrada contribuye a una sola partición de salida
 - Ejemplo: `map()`, `filter()`, `flatMap()`
 - Transformaciones amplias (wide transformations)
 - Particiones de entrada contribuyen a múltiples particiones de salida
 - Requieren intercambio de datos entre nodos (shuffle), lo cual puede ser más costoso en términos de tiempo y recursos

Diferencias clave

- Con las transformaciones estrechas, Spark ejecuta automáticamente una operación llamada pipelining, lo que significa que si especificamos múltiples filtros en los Dataframes, todos se ejecutan en memoria
- Con las transformaciones amplias, al requerirse intercambios de datos entre nodos (shuffles), Spark escribe los resultados en el disco

Acciones

- Las transformaciones no son ejecutadas (se genera un plan de transformación) hasta que se lanza una acción
- La acción más simple es “count”, que da como resultado el numero de registros en el Dataframe
- En Spark, hay tres tipos de acciones:
 - Acciones para ver los data en la consola
 - Acciones para recolectar datos en un objeto nativo
 - Acciones para escribir la salida

Manipulación de datos básica con PySpark

The background of the slide features a woman with dark curly hair and glasses, looking down. The image is partially obscured by large, overlapping geometric shapes in shades of blue, purple, and green. There are also white line patterns resembling stylized paper clips or concentric arcs in the corners.

Dataframes

Los Dataframes de Spark es una estructura de datos tabular, en la cual se pueden almacenar instancias (renglones) que tienen en común un conjunto de descriptores (columnas o atributos)

Los Dataframes de Spark están inspirados en los Dataframes de Pandas, con la gran diferencia de soportar la distribución de particiones sobre un clúster

Id (Int)	First (String)	Last (String)	Url (String)	Published (Date)	Hits (Int)	Campaigns (List[Strings])
1	Jules	Damji	https://tinyurl.1	1/4/2016	4535	[twitter, LinkedIn]
2	Brooke	Wenig	https://tinyurl.2	5/5/2018	8908	[twitter, LinkedIn]
3	Denny	Lee	https://tinyurl.3	6/7/2019	7659	[web, twitter, FB, LinkedIn]
4	Tathagata	Das	https://tinyurl.4	5/12/2018	10568	[twitter, FB]

Tipos de datos soportados (básicos)

Data type	Value assigned in Python	API to instantiate
ByteType	int	DataTypes.ByteType
ShortType	int	DataTypes.ShortType
IntegerType	int	DataTypes.IntegerType
LongType	int	DataTypes.LongType
FloatType	float	DataTypes.FloatType
DoubleType	float	DataTypes.DoubleType
StringType	str	DataTypes.StringType
BooleanType	bool	DataTypes.BooleanType
DecimalType	decimal.Decimal	DecimalType

Tipos de datos soportados (estructurados)

Data type	Value assigned in Python	API to instantiate
BinaryType	bytearray	BinaryType()
TimestampType	datetime.datetime	TimestampType()
DateType	datetime.date	DateType()
ArrayType	List, tuple, or array	ArrayType(dataType, [nullable])
MapType	dict	MapType(keyType, valueType, [nullable])
StructType	List or tuple	StructType([fields])
StructField	A value type corresponding to the type of this field	StructField(name, dataType, [nullable])

Esquemas

- Un esquema en Spark define el nombre de las columnas y los tipos de datos en el Dataframe
- Los esquemas se pueden definir a priori o bien, al momento de leer datos
- En general existen algunas ventajas al definir a priori el esquema que usará un Dataframe
 - Se libera a Spark de definir un esquema a partir de los datos que se leen
 - Se puede detectar errores de tipo de forma temprana si los datos no coinciden con el esquema

Definiendo esquemas

- Alternativa uno: se define el esquema a nivel de programación

```
from pyspark.sql.types import *  
schema = StructType([StructField("author", StringType(), False),  
    StructField("title", StringType(), False),  
    StructField("pages", IntegerType(), False)])
```

Definiendo esquemas

- Alternativa dos: Empleando una cadena DDL (Data Definition Language)

In Python

```
schema = "author STRING, title STRING, pages INT"
```

Creando Dataframes

- Existen diferentes alternativas para crear Dataframes de Spark (para datos estructurados o no estructurados)
 - A partir de un conjunto de datos que se importa de un archivo
 - Creando el Dataframe a partir de un esquema y conjunto de datos iniciales

spark.read.csv

- Comando usado para crear un Dataframe a partir de un archivo csv
- Parámetros
 - Por defecto, el parámetro “**header**” está puesto como verdadero, lo que indica que el primer renglón del archivo será tomado como encabezado
 - Por defecto también, todas las columnas son leídas como StringType, pero al poner como verdadero el valor “**inferSchema**” se deriva el verdadero tipo de datos de las columnas y la estructura de los datos, lo que crea un conjunto de datos Spark SQL
 - **sep** = “ ”, parámetro donde se define el tipo de separador usado en el archivo csv

spark.read.csv

```
df = spark.read.csv('cars.csv', header=True, sep=";", inferSchema=True)
df.show(10)
```

Car	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model	Origin
Chevrolet Chevelle	18.0	8	307.0	130.0	3504	12.0	70	US
Buick Skylark 320	15.0	8	350.0	165.0	3693	11.5	70	US
Plymouth Satellite	18.0	8	318.0	150.0	3436	11.0	70	US
AMC Rebel SST	16.0	8	304.0	150.0	3433	12.0	70	US
Ford Torino	17.0	8	302.0	140.0	3449	10.5	70	US
Ford Galaxie 500	15.0	8	429.0	198.0	4341	10.0	70	US
Chevrolet Impala	14.0	8	454.0	220.0	4354	9.0	70	US
Plymouth Fury iii	14.0	8	440.0	215.0	4312	8.5	70	US
Pontiac Catalina	14.0	8	455.0	225.0	4425	10.0	70	US
AMC Ambassador DPL	15.0	8	390.0	190.0	3850	8.5	70	US

only showing top 10 rows

Dataframe a partir de un DDL

- Se crea un esquema a partir de un DDL

```
# Se especifica un DDL
ddl_schema = "Car STRING, MPG FLOAT, Cylinders INT, Displacement FLOAT, Horsepower FLOAT, Weight FLOAT, Acceleration FLOAT, Model INT, Origin STRING"

df3 = spark.read.csv('cars.csv', header=True, sep=";", schema=ddl_schema)
df3.printSchema()
```

```
root
|-- Car: string (nullable = true)
|-- MPG: float (nullable = true)
|-- Cylinders: integer (nullable = true)
|-- Displacement: float (nullable = true)
|-- Horsepower: float (nullable = true)
|-- Weight: float (nullable = true)
|-- Acceleration: float (nullable = true)
|-- Model: integer (nullable = true)
|-- Origin: string (nullable = true)
```

Operaciones en registros y columnas con PySpark



Operaciones en Dataframes

Transformaciones Columna

- `Select()`
- `withColumn()`
- `withColumnRenamed()`
- `groupBy()`
- `drop()`

Transformaciones Registro

- `filter()`
- `distinct()`
- `orderBy()`
- `union()`

Columnas: select()

Selección de columnas. Como parámetro se define la columna o columnas a extraer. Como resultado se obtiene un nuevo Dataframe

Sintaxis:

```
dataframe.select("columnName")  
dataframe.select([column1,  
column2, ...])
```

```
# Para la seleccion de columnas de un  
print("Método 1")  
df_car = df.select(df.Car)  
df_car.show(10, truncate=False)  
  
print("Método 2")  
df_car = df.select(df['Car'])  
df_car.show(10, truncate=False)  
  
print("Método 3")  
df_car = df.select('Car')  
df_car.show(10, truncate=False)
```

Columnas: select()

Para recuperar varias columnas de un Dataframe, se debe de especificar la secuencia de columnas a recuperar

```
# Es posible seleccionar varias columnas en una sola transformacion
# Nota: se puede mezclar sintaxis

# Seleccion de columna "Car" y "MPG"
# Método 1
print("Método 1")
df_Car_MPG = df.select(df.Car, df.MPG)
df_Car_MPG.show(10,truncate=False)

# Método 2
print("Método 2")
df_Car_MPG = df.select(df['Car'], df.MPG)
df_Car_MPG.show(10,truncate=False)

# Método 3
print("Método 3")
from pyspark.sql.functions import col
df_Car_MPG = df.select(col('car'), col('MPG'))
df_Car_MPG.show(10,truncate=False)
```

Columnas: withColumn

- Función que retorna un nuevo Dataframe añadiendo una nueva columna, o bien reemplazando una columna existente si tiene el mismo nombre

```
# Caso 1: Añadiendo la columna "col_1"
from pyspark.sql.functions import lit
df_newCols = df.withColumn('col_1',lit(1))
# lit se usa para especificar el valor a usar para llenar la columna creada

# Caso 2: Añadiendo dos columnas
df_newCols = df_newCols.withColumn('col_2', lit(2)) \
                        .withColumn('col_3', lit('c3'))
df_newCols.show(10,truncate=False)
```

Columnas: withColumn

Con este operador, es posible generar un nuevo Dataframe modificado el esquema de un Dataframe original

```
# Se debe recordar que withColumn también es posible cambiar el tipo de dato de una columna
# Dos formas en las cuales se puede cambiar el formato de la columna MPG
df_newCols.printSchema()
```

```
df_newCols = df_newCols.withColumn('col_1', df_newCols['col_1'].cast('string'))
```

```
df_newCols.printSchema()
```

```
root
|-- Car: string (nullable = true)
|-- MPG: double (nullable = true)
|-- Cylinders: integer (nullable = true)
|-- Displacement: double (nullable = true)
|-- Horsepower: double (nullable = true)
|-- Weight: decimal(4,0) (nullable = true)
|-- Acceleration: double (nullable = true)
|-- Model: integer (nullable = true)
|-- Origin: string (nullable = true)
|-- col_1: integer (nullable = false)
|-- col_2: integer (nullable = false)
|-- col_3: string (nullable = false)
|-- car_model: string (nullable = true)
```

```
root
|-- Car: string (nullable = true)
|-- MPG: double (nullable = true)
|-- Cylinders: integer (nullable = true)
|-- Displacement: double (nullable = true)
|-- Horsepower: double (nullable = true)
|-- Weight: decimal(4,0) (nullable = true)
|-- Acceleration: double (nullable = true)
|-- Model: integer (nullable = true)
|-- Origin: string (nullable = true)
|-- col_1: string (nullable = false)
|-- col_2: integer (nullable = false)
|-- col_3: string (nullable = false)
|-- car_model: string (nullable = true)
```


Columnas: withColumnRenamed

- Comando usado para renombra columnas
- Sintaxis: dataframe.withColumnRenamed(oldNameCol, NewNameCol)

```
df_newCols = df_newCols.withColumnRenamed('col_1', 'col1') \
    .withColumnRenamed('col_2', 'col2') \
    .withColumnRenamed('col_3', 'col3')
df_newCols.show(10,truncate=False)
```

Car	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model	Origin	col1	col2	col3	car_model
Chevrolet Chevelle Malibu	18.0	8	307.0	130.0	3504	12.0	70	US	1	2	c3	Chevrolet Chevelle Malibu 70
Buick Skylark 320	15.0	8	350.0	165.0	3693	11.5	70	US	1	2	c3	Buick Skylark 320 70
Plymouth Satellite	18.0	8	318.0	150.0	3436	11.0	70	US	1	2	c3	Plymouth Satellite 70
AMC Rebel SST	16.0	8	304.0	150.0	3433	12.0	70	US	1	2	c3	AMC Rebel SST 70
Ford Torino	17.0	8	302.0	140.0	3449	10.5	70	US	1	2	c3	Ford Torino 70
Ford Galaxie 500	15.0	8	429.0	198.0	4341	10.0	70	US	1	2	c3	Ford Galaxie 500 70
Chevrolet Impala	14.0	8	454.0	220.0	4354	9.0	70	US	1	2	c3	Chevrolet Impala 70
Plymouth Fury iii	14.0	8	440.0	215.0	4312	8.5	70	US	1	2	c3	Plymouth Fury iii 70
Pontiac Catalina	14.0	8	455.0	225.0	4425	10.0	70	US	1	2	c3	Pontiac Catalina 70
AMC Ambassador DPL	15.0	8	390.0	190.0	3850	8.5	70	US	1	2	c3	AMC Ambassador DPL 70

only showing top 10 rows

Columnas: groupBy

Operador que permite la agrupación de valores a partir de las columnas especificadas y se aplica una función de agregación

Sintaxis:

`dataframe.groupBy(column).func`

func es la función de agregación:
`count()`, `avg()`, `max()`, `min()`

```
# La funcion groupBy agrupa los valores de una columna y se debe indicar
# una función de agregación para saber que se va a realizar con dichos valores
# Funciones de agregación: avg, count, max, min
# Caso 1: agrupando valores a partir de una columna
print("Agrupando a partir de una sola columna")
df_grouped = df.groupBy('Origin').count()
df_grouped.show(5)

# Caso 2: agrupando valores a partir de dos columnas
df_grouped = df.groupBy('Origin', 'Model').count()
df_grouped.show(5)
```

Agrupando a partir de una sola columna

Origin	count
Europe	73
US	254
Japan	79

Origin	Model	count
Europe	71	5
Europe	80	9
Europe	70	4

Columnas: drop

Permite eliminar una columna de un Dataframe

Sintaxis: `dataframe.drop(columnName)`

```
# se eliminan las columnas del dataframe
```

```
df_newCols.show(10,truncate=False)
```

```
df_newCols1 = df_newCols.drop('col1').drop('col2').drop('col3')
```

```
df_newCols1.show(10,truncate=False)
```

Car	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model	Origin	col1	col2	col3	car_model
Chevrolet Chevelle Malibu	18.0	8	307.0	130.0	3504	12.0	70	US	1	2	c3	Chevrolet Chevelle Malibu 70
Buick Skylark 320	15.0	8	350.0	165.0	3693	11.5	70	US	1	2	c3	Buick Skylark 320 70
Plymouth Satellite	18.0	8	318.0	150.0	3436	11.0	70	US	1	2	c3	Plymouth Satellite 70
AMC Rebel SST	16.0	8	304.0	150.0	3433	12.0	70	US	1	2	c3	AMC Rebel SST 70
Ford Torino	17.0	8	302.0	140.0	3449	10.5	70	US	1	2	c3	Ford Torino 70
Ford Galaxie 500	15.0	8	429.0	198.0	4341	10.0	70	US	1	2	c3	Ford Galaxie 500 70
Chevrolet Impala	14.0	8	454.0	220.0	4354	9.0	70	US	1	2	c3	Chevrolet Impala 70
Plymouth Fury iii	14.0	8	440.0	215.0	4312	8.5	70	US	1	2	c3	Plymouth Fury iii 70
Pontiac Catalina	14.0	8	455.0	225.0	4425	10.0	70	US	1	2	c3	Pontiac Catalina 70
AMC Ambassador DPL	15.0	8	390.0	190.0	3850	8.5	70	US	1	2	c3	AMC Ambassador DPL 70
only showing top 10 rows												

Car	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model	Origin	car_model
Chevrolet Chevelle Malibu	18.0	8	307.0	130.0	3504	12.0	70	US	Chevrolet Chevelle Malibu 70
Buick Skylark 320	15.0	8	350.0	165.0	3693	11.5	70	US	Buick Skylark 320 70
Plymouth Satellite	18.0	8	318.0	150.0	3436	11.0	70	US	Plymouth Satellite 70
AMC Rebel SST	16.0	8	304.0	150.0	3433	12.0	70	US	AMC Rebel SST 70
Ford Torino	17.0	8	302.0	140.0	3449	10.5	70	US	Ford Torino 70
Ford Galaxie 500	15.0	8	429.0	198.0	4341	10.0	70	US	Ford Galaxie 500 70
Chevrolet Impala	14.0	8	454.0	220.0	4354	9.0	70	US	Chevrolet Impala 70
Plymouth Fury iii	14.0	8	440.0	215.0	4312	8.5	70	US	Plymouth Fury iii 70
Pontiac Catalina	14.0	8	455.0	225.0	4425	10.0	70	US	Pontiac Catalina 70
AMC Ambassador DPL	15.0	8	390.0	190.0	3850	8.5	70	US	AMC Ambassador DPL 70

only showing top 10 rows

Renglón: filtered

Operador que permite recuperar un conjunto de registros a partir de un filtro especificado

Sintaxis: dataframe.filtered(condicion)

```
# Ejemplo 1: recuperar todos los automóviles hechos en Japon
df_japan_filtered = df.filter(df['Origin']=='Japan')
print("Total de registros origen Japones : " + str(df_japan_filtered.count()))
df_japan_filtered.show(10, truncate=False)
```

Total de registros origen Japones : 79

Car	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model	Origin
Toyota Corolla Mark ii	24.0	4	113.0	95.0	2372	15.0	70	Japan
Datsun PL510	27.0	4	97.0	88.0	2130	14.5	70	Japan
Datsun PL510	27.0	4	97.0	88.0	2130	14.5	71	Japan
Toyota Corolla	25.0	4	113.0	95.0	2228	14.0	71	Japan
Toyota Corolla 1200	31.0	4	71.0	65.0	1773	19.0	71	Japan
Datsun 1200	35.0	4	72.0	69.0	1613	18.0	71	Japan
Toyota Corolla Hardtop	24.0	4	113.0	95.0	2278	15.5	72	Japan
Mazda RX2 Coupe	19.0	3	70.0	97.0	2330	13.5	72	Japan
Datsun 510 (sw)	28.0	4	97.0	92.0	2288	17.0	72	Japan
Toyota Corolla Mark II (sw)	23.0	4	120.0	97.0	2506	14.5	72	Japan

only showing top 10 rows

Operadores

- Es importante recordar los operadores lógicos que se pueden definir en Python
 - And (&): combina condiciones dónde todas deben ser verdaderas
 - Or (|): combina múltiples condiciones en las que al menos una es verdadera
 - Not (~): invierte la condición, es decir, selecciona las filas donde la condición no es verdadera
 - isNull(), isNotNull(): verifica si una columna es NULL o no lo es
 - ==: compara la igualdad de valores
 - eqNullSafe(): para casos donde los valores pueden ser NULL
 - >, <, >=, <=: comparación entre valores

Renglones: distinct

Función que permite eliminar registros o renglones duplicados

Sintaxis: dataframe.distinct()

```
# Eliminando registros duplicados

df_origin_model = df.select(df.Origin, df.Model).distinct()
print("Total de registros en Dataframe origin_model: " + str(df_origin_model.count()))
df_origin_model.show(10, truncate=False)
# Si no se hubieran eliminado registros repetidos
print("Total de registros en Dataframe origin_model con duplicados: " + str(df.select(df.Origin, df.Model).count()))
```

Total de registros en Dataframe origin_model: 39

Origin	Model
Europe	71
Europe	80
Europe	79
Japan	75
US	72
US	80
Europe	74
Japan	79
Europe	76
US	75

only showing top 10 rows

Total de registros en Dataframe origin_model con duplicados: 406

Renglones: orderBy

Permite ordenar registros a partir de un atributo o atributos especificados

Sintaxis: dataframe.orderBy(atributo, orden)

orden: puede tomar los valores
ascending = True / False

Ejemplo donde se ordenan un conjunto de registros a partir de un atributo específico

```
df_Origin_Model_sort = df.select(df.Origin, df.Model).distinct().orderBy('Model', ascending=False)  
df_Origin_Model_sort.show(10,truncate=False)
```

```
+-----+-----+  
|Origin|Model|  
+-----+-----+  
|Europe|82  |  
|Japan  |82  |  
|US     |82  |  
|Japan  |81  |  
|US     |81  |  
|Europe|81  |  
|Japan  |80  |  
|Europe|80  |  
|US     |80  |  
|Europe|79  |  
+-----+-----+  
only showing top 10 rows
```

Unión de Dataframes

Es posible realizar la unión de dos o más dataframes a partir del operador `union()`

Sintaxis: `dataframe.unión(dataframe2)`

```
# se crean dos dataframes diferentes con igual esquema
df_japan_4cil = df.filter((df.Origin == 'Japan') & (df.Cylinders == 4))
df_japan_6cil = df.filter((df.Origin == 'Japan') & (df.Cylinders == 6))
# Se unen los dataframes
df_japan_4cil_6cil = df_japan_4cil.union(df_japan_6cil)
df_japan_4cil_6cil.show(10,truncate = False)
df_japan_4cil_6cil.count()
```

Car	MPG	Cylinders	Displacement	Horsepower	Weight
Toyota Corolla Mark ii	24.0	4	113.0	95.0	2372
Datsun PL510	27.0	4	97.0	88.0	2130
Datsun PL510	27.0	4	97.0	88.0	2130
Toyota Corolla	25.0	4	113.0	95.0	2228
Toyota Corolla 1200	31.0	4	71.0	65.0	1773
Datsun 1200	35.0	4	72.0	69.0	1613
Toyota Corolla Hardtop	24.0	4	113.0	95.0	2278
Datsun 510 (sw)	28.0	4	97.0	92.0	2288
Toyota Corolla Mark II (sw)	23.0	4	120.0	97.0	2506
Toyota Corolla 1600 (sw)	27.0	4	97.0	88.0	2100

only showing top 10 rows

SQL en Spark

The image features a woman with dark curly hair and glasses, looking down. She is wearing a light-colored top and a thin necklace. The background is a mix of dark blue and green tones with abstract geometric shapes, including a large dark blue circle on the left and several diagonal bands of different colors (dark blue, green, purple, grey) on the right. There are also white line patterns in the bottom left and top left corners.

SQL en pySpark

El módulo de SQL de Spark permite usar la sintaxis definida en el lenguaje de consulta SQL

1. Primero se tiene que registrar el dataframe a través de un “alias”
 1. `Dataframe.createOrReplaceTempView('alias')`
2. Definir los queries a través de `spark.sql`
 1. `spark.sql (query)`

SQL en Spark

- Los **queries simples** se pueden encasillar con **comillas dobles** o **comillas sencillas**
- Queries con una mayor estructura se encasillan en **triple comilla**
- Esto se hace para utilizar las comillas dobles o comillas sencillas para **valores constantes texto** y usar las comillas invertidas para **columnas que tienen espacios en blanco**

SQL Spark

```
# Se registra temporalmente la tabla con el nombre "temp"
df.createOrReplaceTempView("temp")
# Se imprimen los 5 primeros registros
spark.sql("select * from temp limit 5").show()
# Se calcula el total de registros de la tabla
spark.sql("select count(*) as total_count from temp").show()
```

Car	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model	Origin
Chevrolet Chevelle	18.0	8	307.0	130.0	3504	12.0	70	US
Buick Skylark 320	15.0	8	350.0	165.0	3693	11.5	70	US
Plymouth Satellite	18.0	8	318.0	150.0	3436	11.0	70	US
AMC Rebel SST	16.0	8	304.0	150.0	3433	12.0	70	US
Ford Torino	17.0	8	302.0	140.0	3449	10.5	70	US

total_count
406

Conclusiones



Conclusiones Generales

- Spark a través de su versión para programación en Python (PySpark) ofrece una interfaz de programación con un alto nivel de transparencia
- Las transformaciones se deben de diseñar de tal forma que PySpark pueda optimizar las transformaciones, antes de lanzar acciones
- Los dataframes son estructuras de datos distribuidas que permite programación de alto nivel
- La interfaz de desarrollo para la manipulación de Dataframes es flexible, con comandos propios de PySpark así como de SQL



D.R. © Tecnológico de Monterrey, México, 2024.
Prohibida la reproducción total o parcial
de esta obra sin expresa autorización del
Tecnológico de Monterrey.